

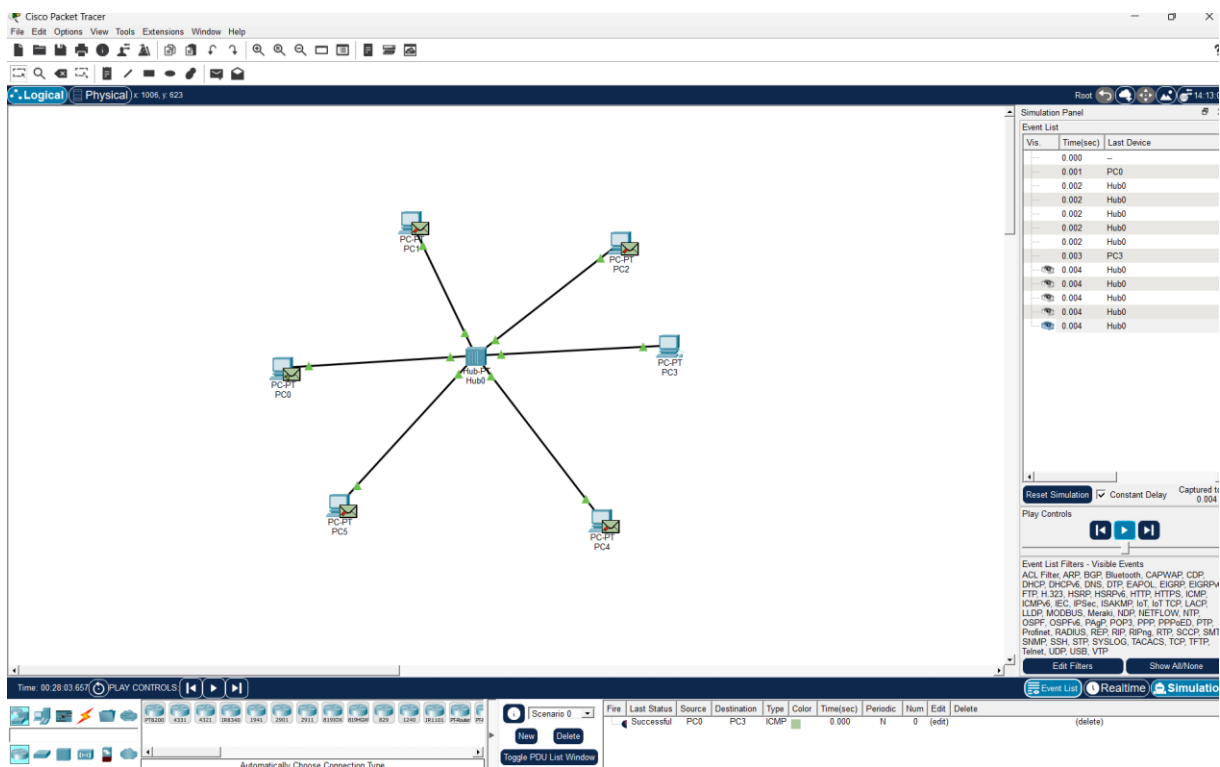
Name: Y. Varun Kumar Reddy

Reg No: 192424230

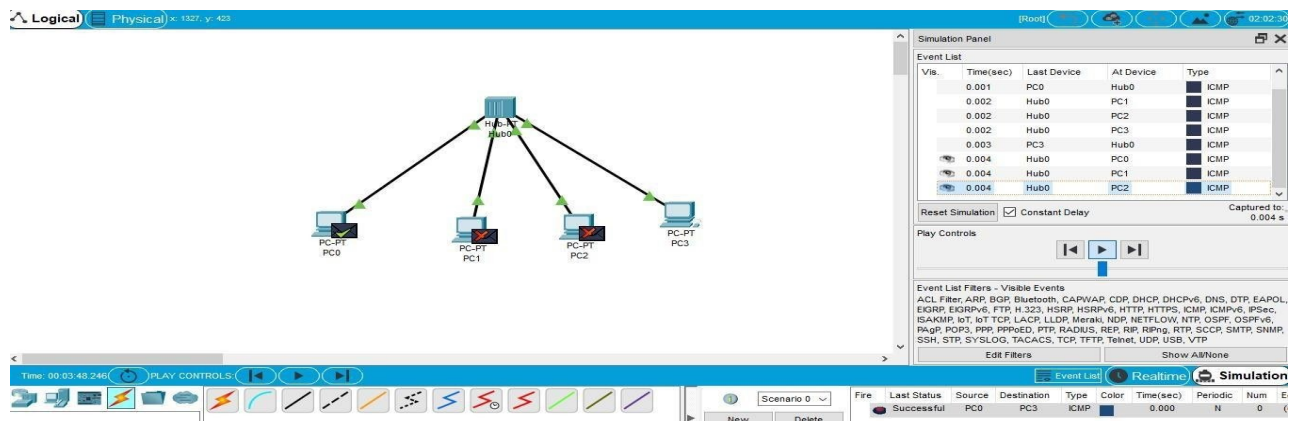
**Course Name: Computer Networks for
Software Application**

Code: CSA0713

**1.Configuration of Network Devices using Packet Tracer tools (Hub,
Switch, Ethernet, Broadcast)**

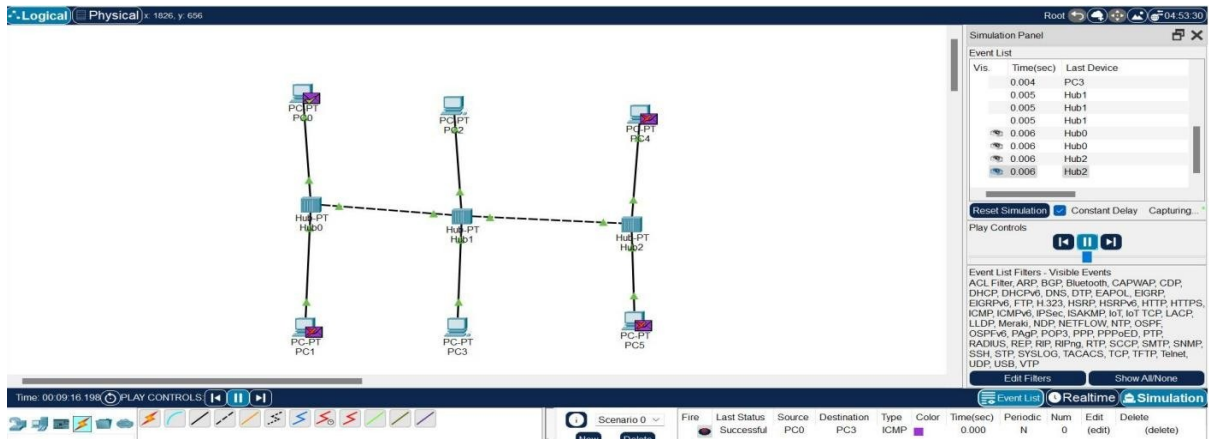


2. Design and Configuration of Star Topologies using Packet Tracer Star Topology using Hub:



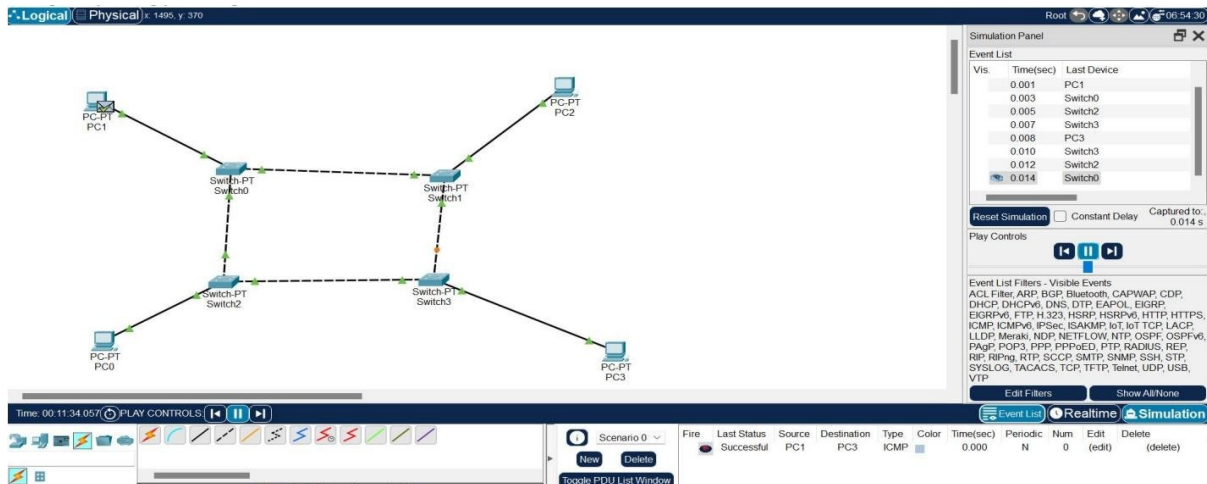
3. Design and Configuration of BUS Topologies using Packet Tracer.

Bus Topology using Hub:



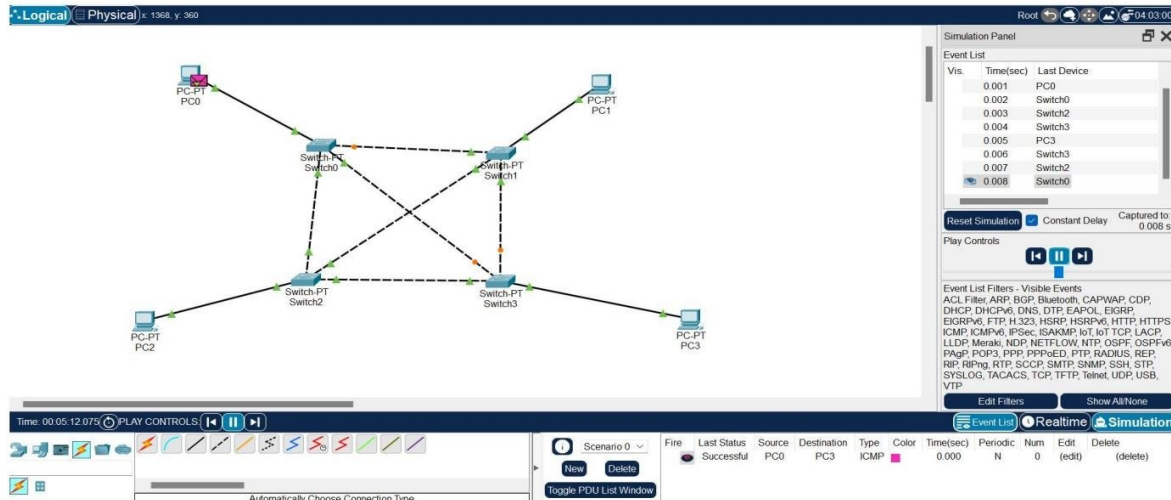
4. Design and Configuration of RING Topologies using Packet Tracer

Ring Topology using Switch:



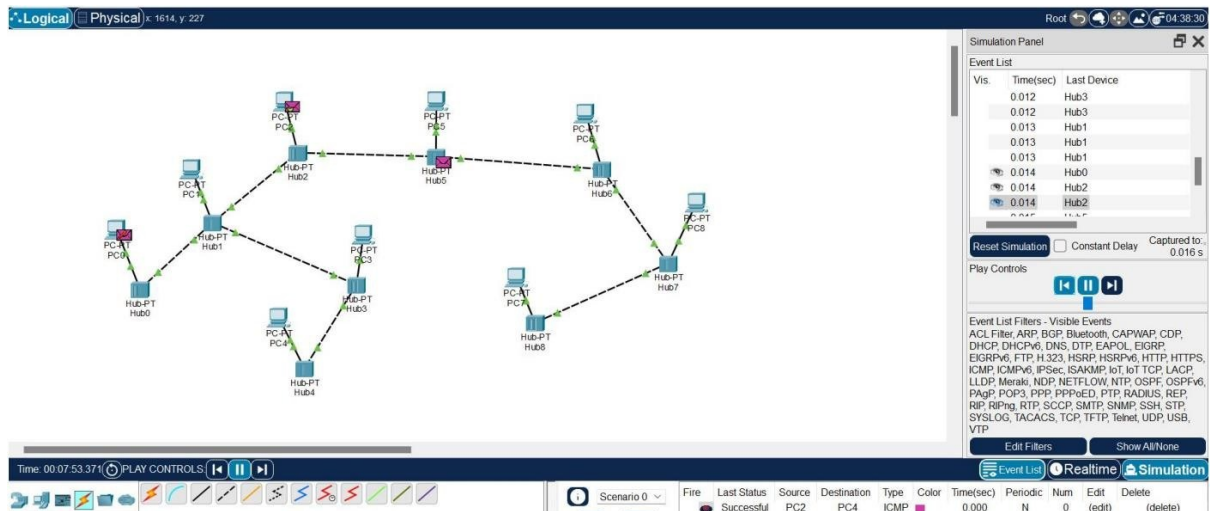
5. Design and Configuration of Mesh Topologies using Packet Tracer

Mesh Topology using Switch:



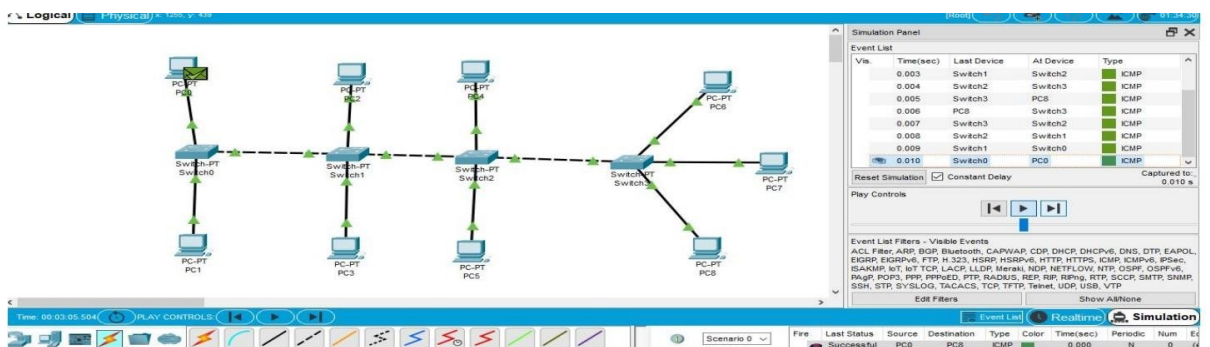
6.Design and Configuration of Tree Topologies using Packet Tracer.

Tree Topology using Hub:

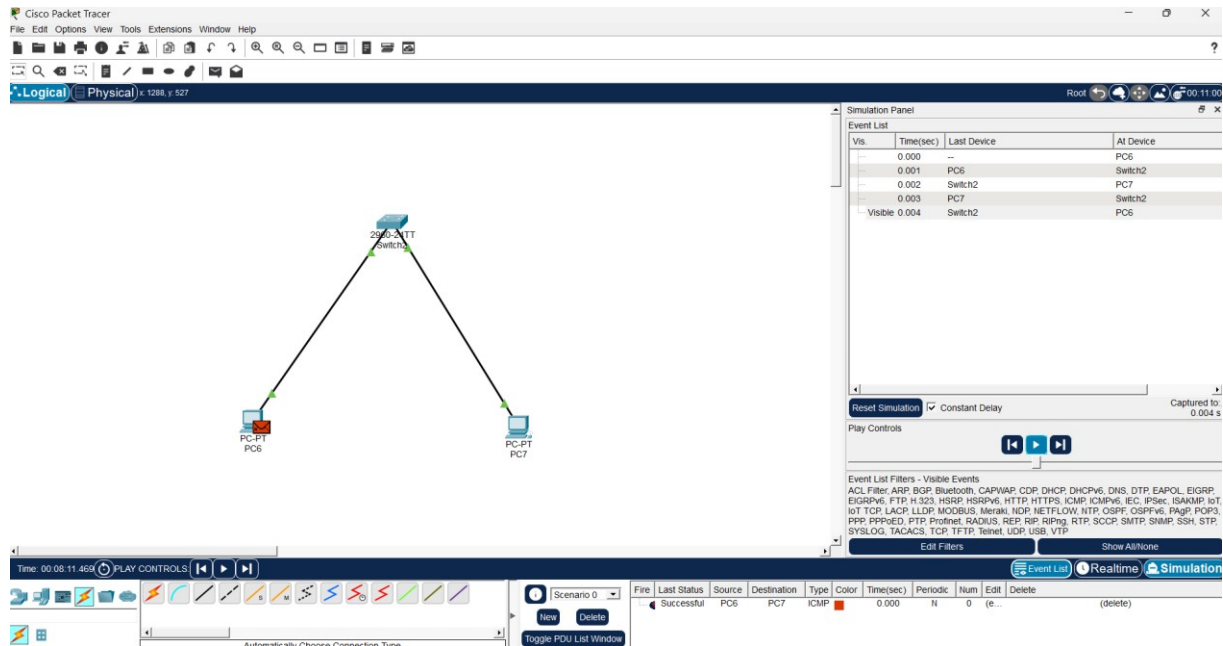


7.Design and Configuration of Hybrid Topologies using Packet Tracer.

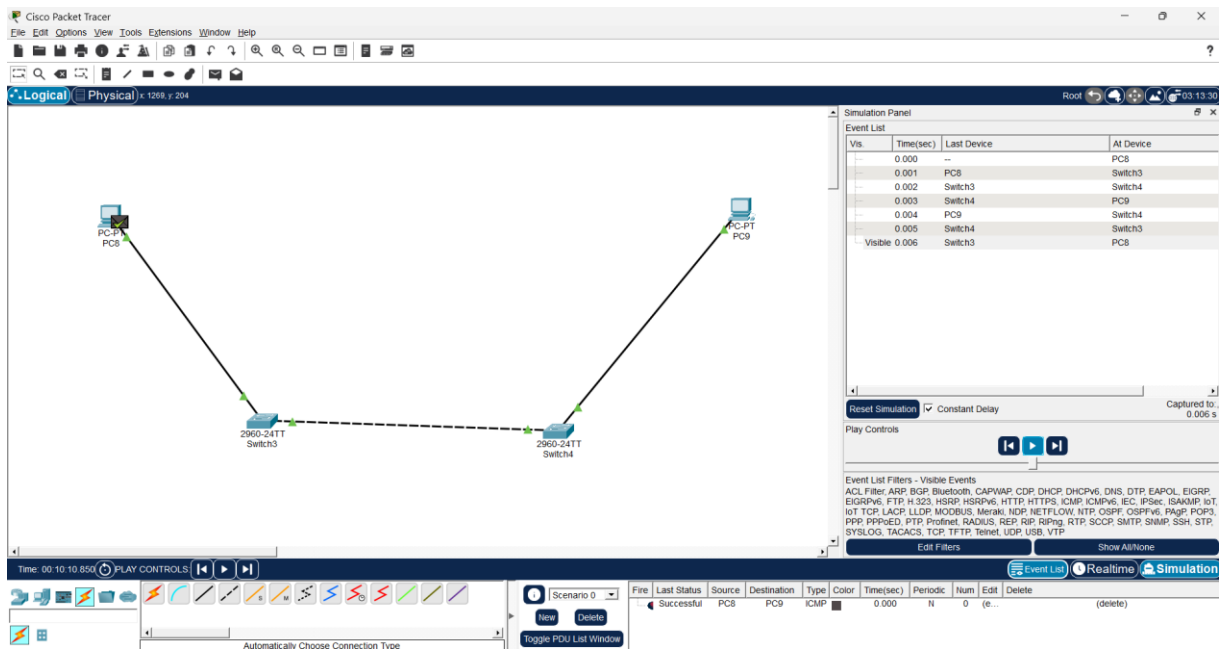
Hybrid Topology using Switch:



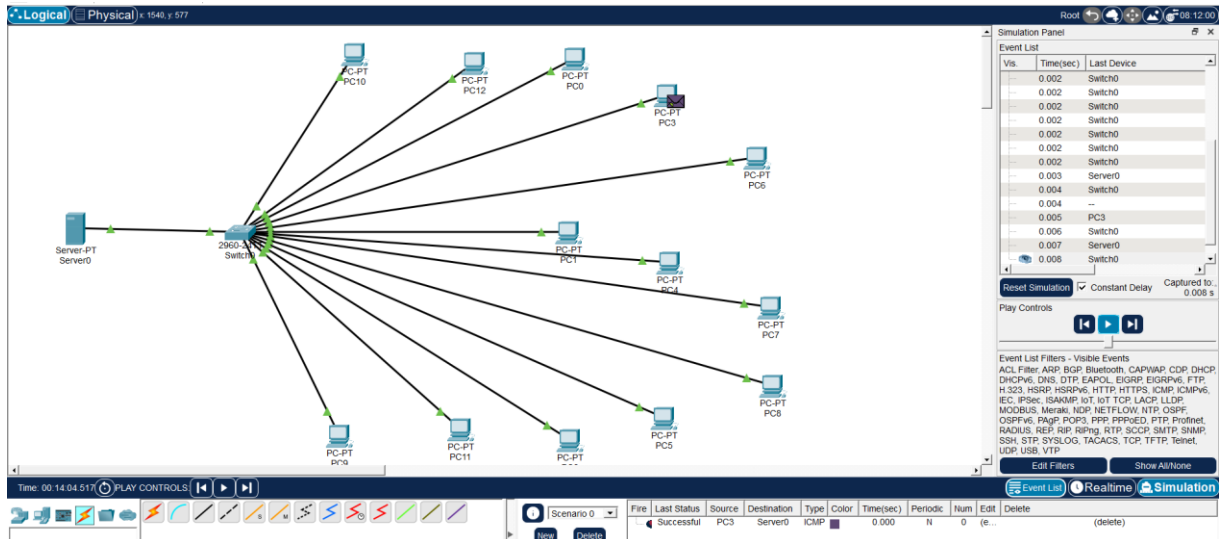
8.Data Link Layer Traffic Simulation using Packet Tracer Analysis of ARP.



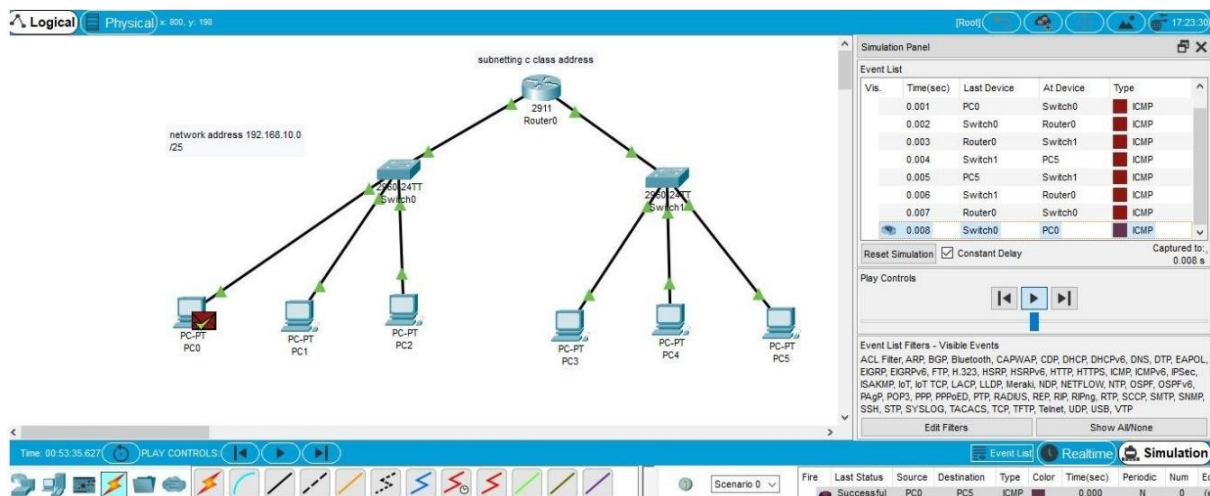
9.Data Link Layer Traffic Simulation using Packet Tracer Analysis of LLDP.



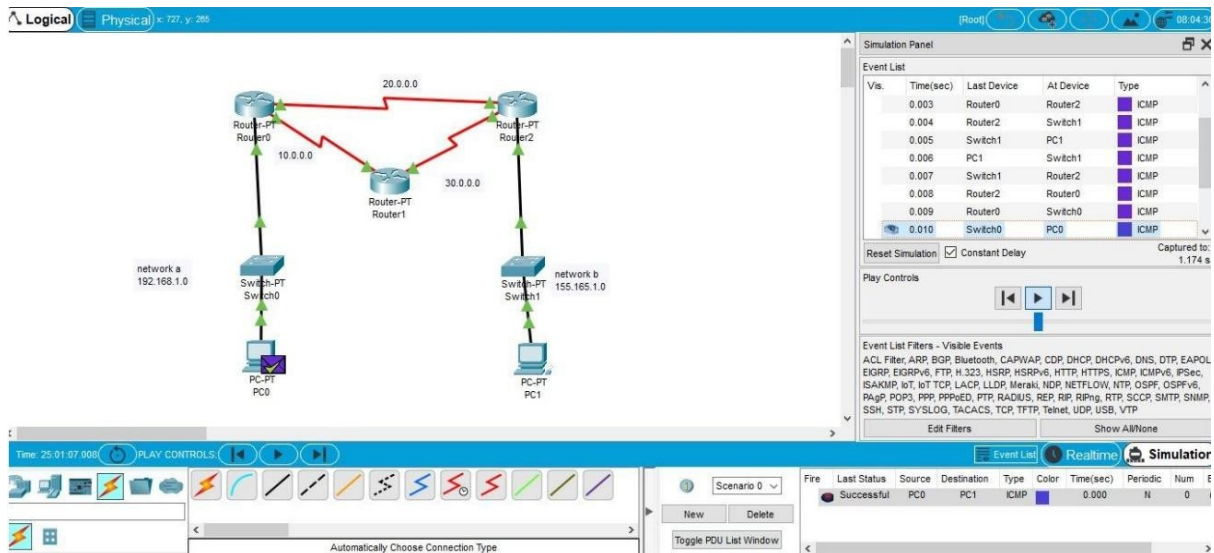
10. Making Computer Lab in Cisco Packet Tracer.



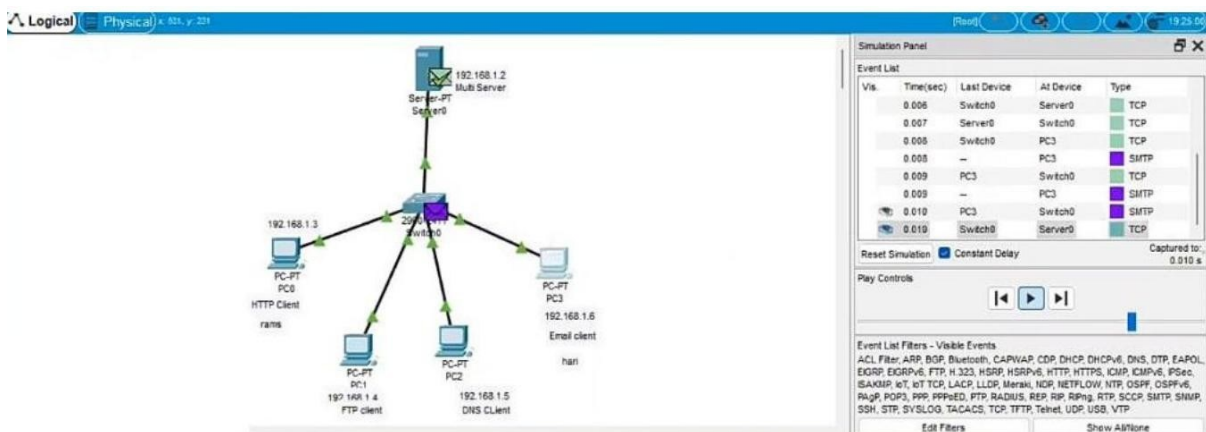
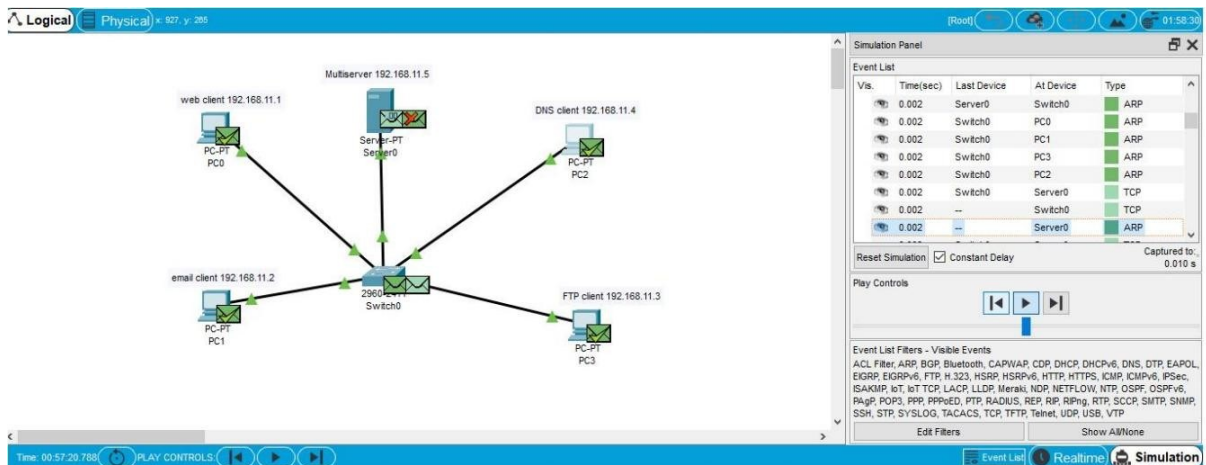
11. Designing two different network with Static Routing techniques using Packet Tracer.



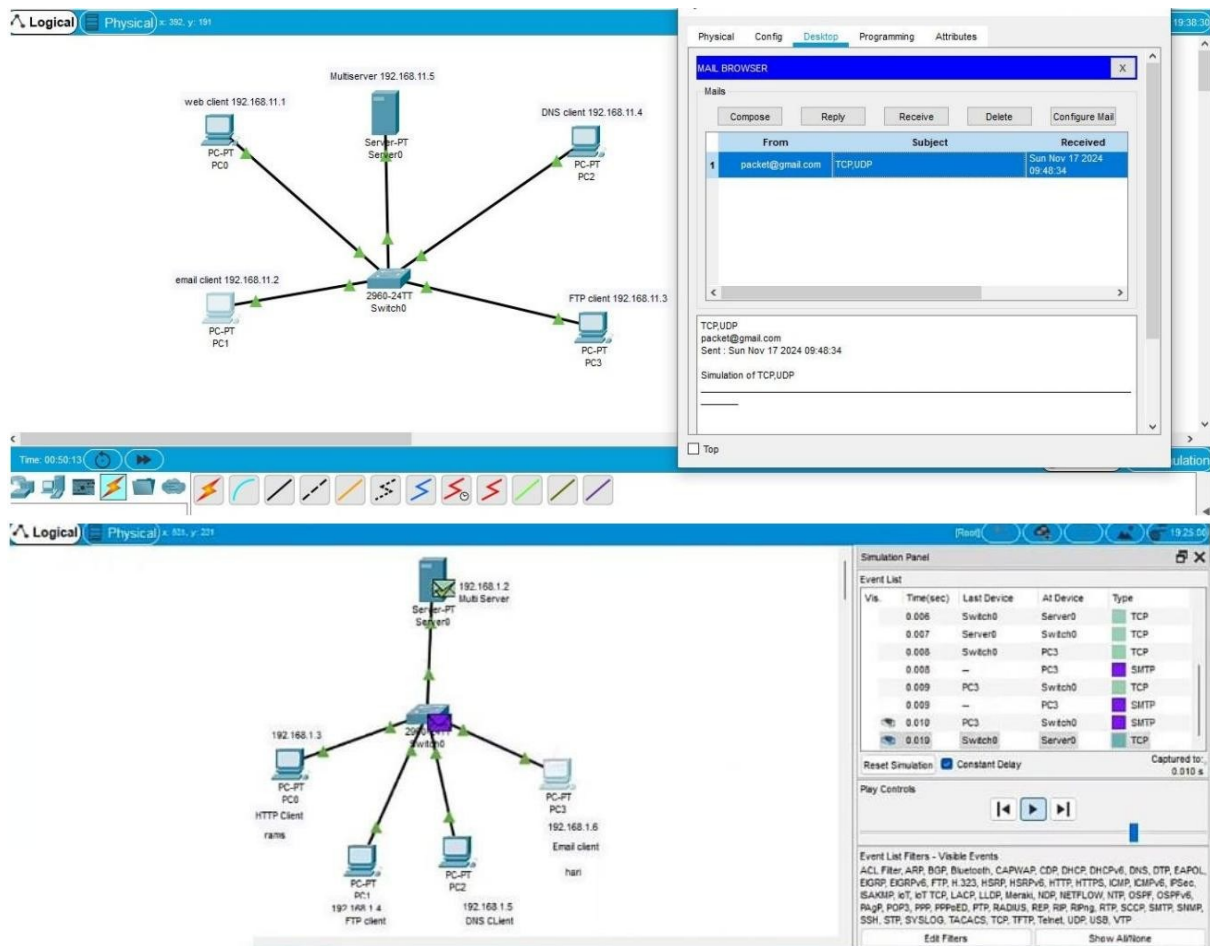
12. Design the Functionalities and Exploration of TCP using Packet Tracer.



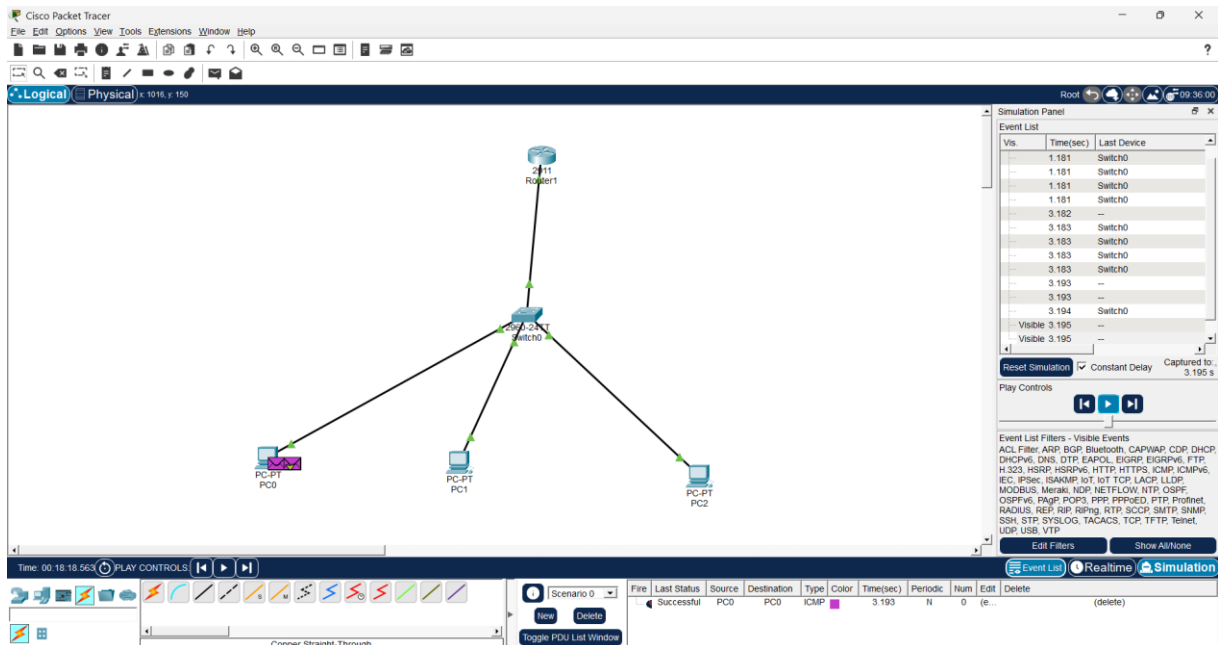
13. Design the network model for Subnetting – Class C Addressing using Packet Tracer.



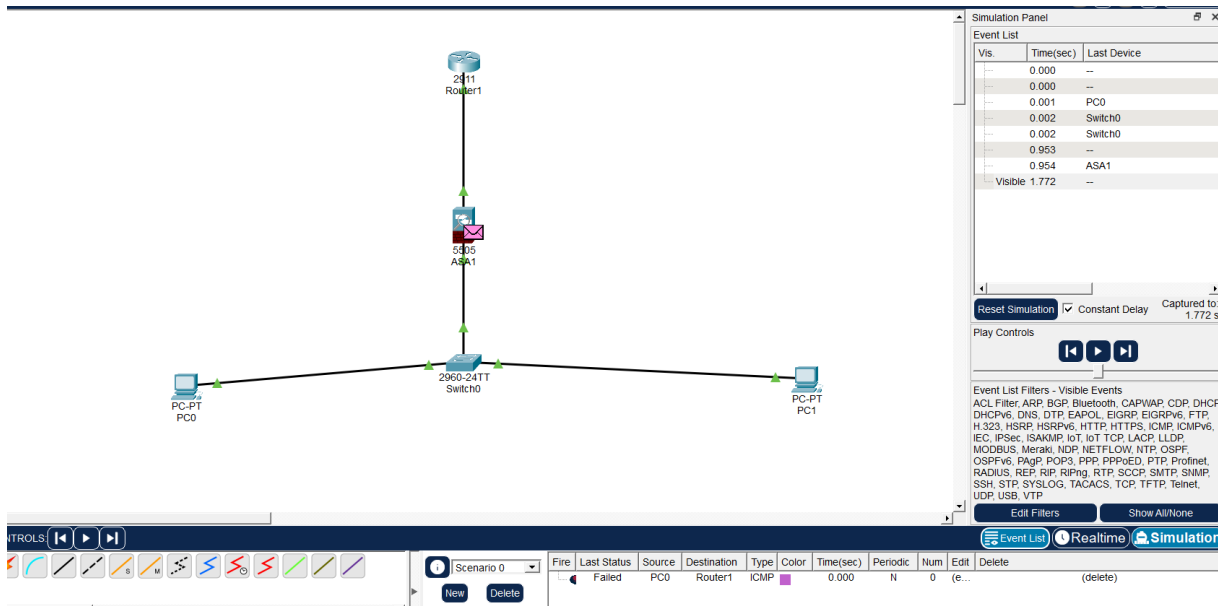
14. Simulating X, Y, Z Company Network Design and simulate using Packet Tracer.



15. Configuration of DHCP (dynamic host configuration protocol) in packet Tracer.

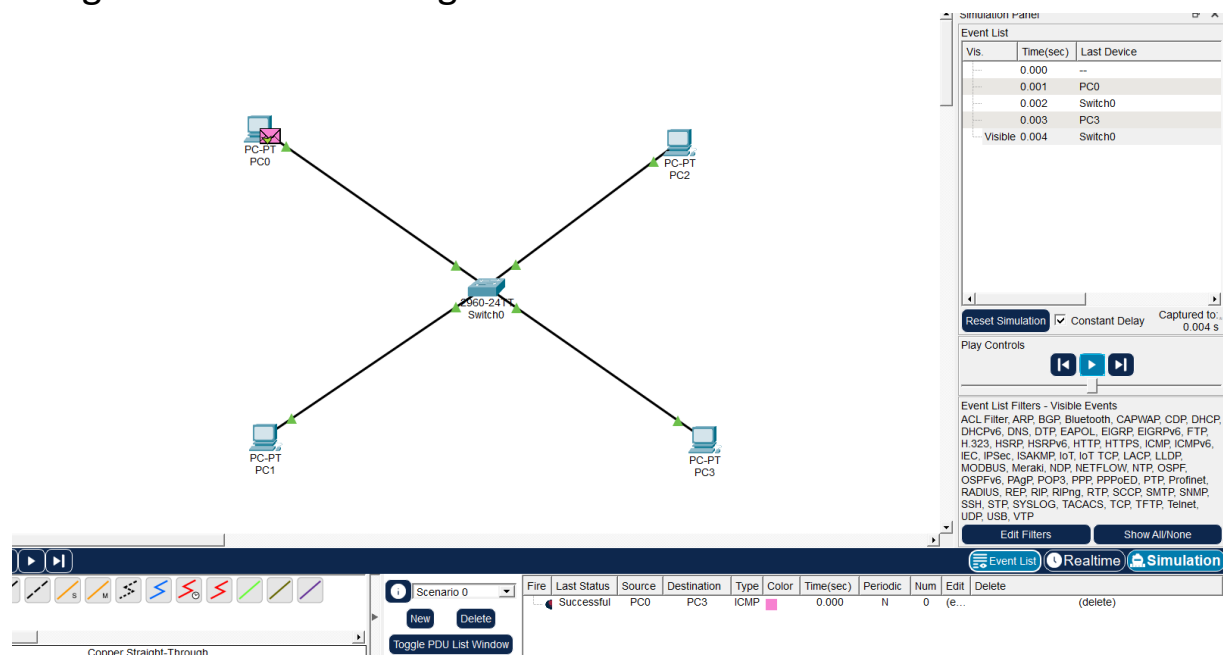


16. Configuration of firewall in packet tracer.

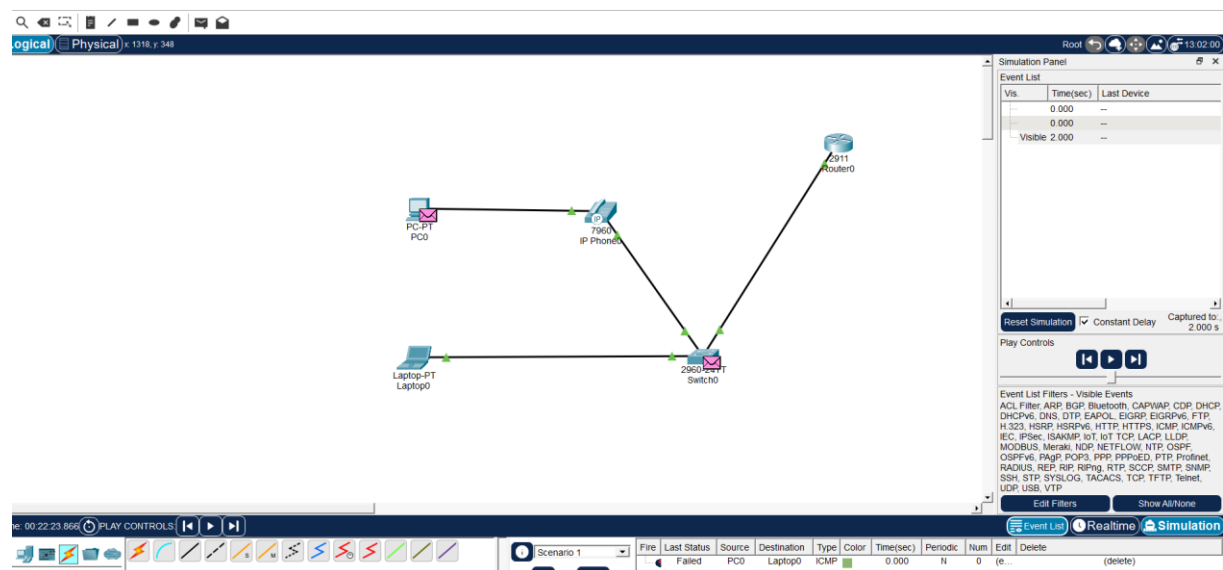


17. Make a Computer Lab to transfer a message from one node to another to

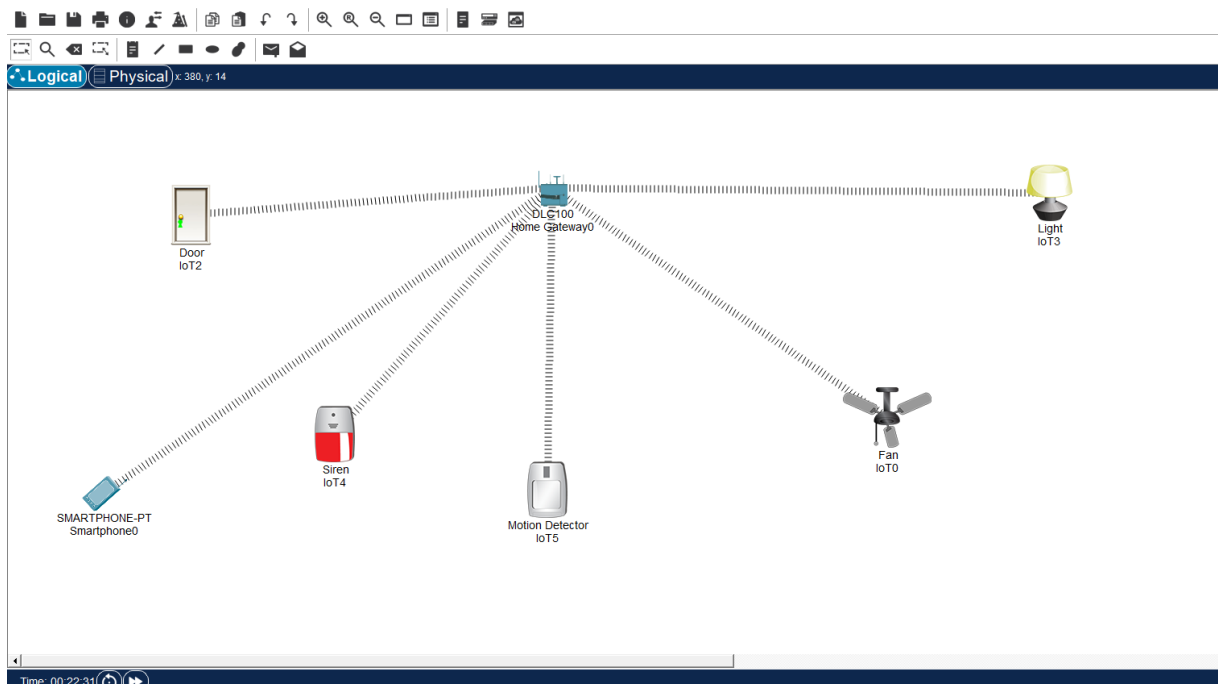
design and simulate using Cisco Packet Tracer.



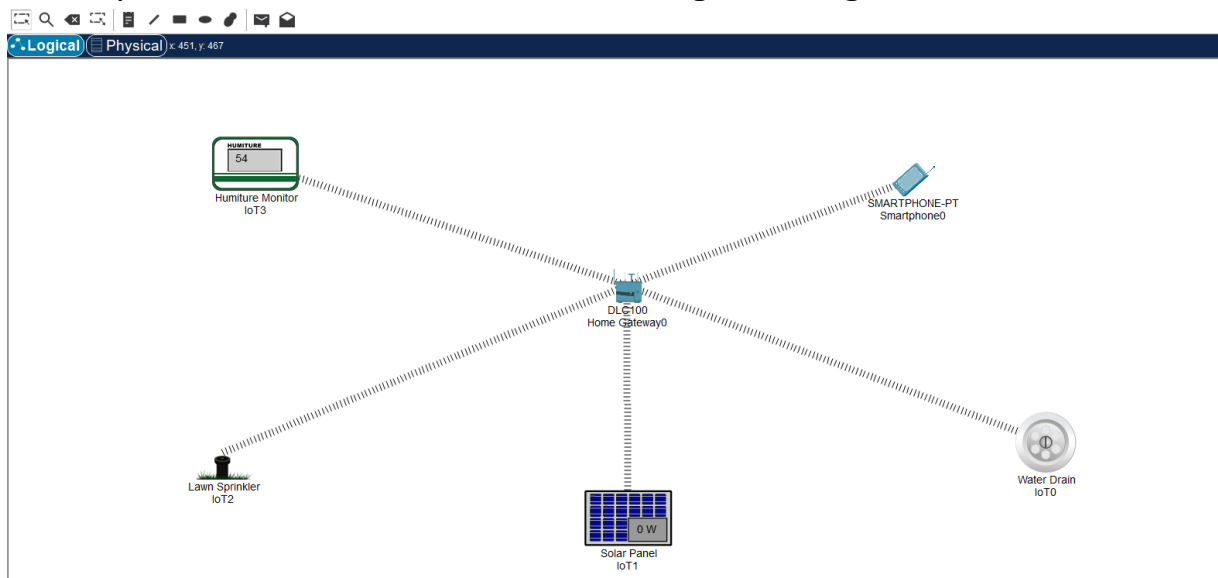
18. Simulate a Multimedia Network in Cisco Packet Tracer.



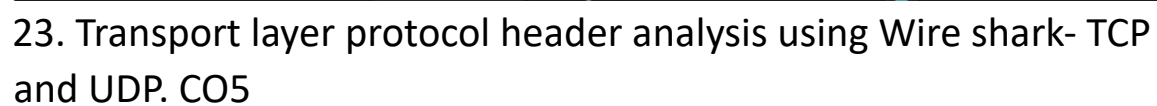
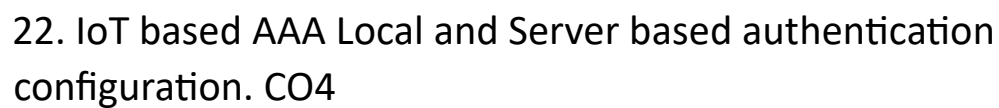
19. IoT based smart home applications. CO3



20. Implementation of IoT based smart gardening. CO2



21. Implementation of IoT devices in networking. CO4



udp

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	172.23.22.137	224.0.0.251	MDNS	77	Standard query 0x0000 PTR _dosvc._tcp.local, "QU" question
2	0.000001600	fe80::27e6:aa7b:bbf...	ff02::fb	MDNS	97	Standard query 0x0000 PTR _dosvc._tcp.local, "QU" question
3	0.317055500	172.23.17.236	255.255.255.255	UDP	125	10004 → 10004 Len=83
4	0.317056500	172.23.17.236	255.255.255.255	UDP	125	10004 → 10004 Len=83
5	0.317057600	172.23.16.70	239.255.255.250	SSDP	441	NOTIFY * HTTP/1.1
6	0.317058500	172.23.16.70	239.255.255.250	SSDP	513	NOTIFY * HTTP/1.1
7	0.317059400	172.23.16.70	239.255.255.250	SSDP	450	NOTIFY * HTTP/1.1
8	0.317060200	172.23.16.70	239.255.255.250	SSDP	509	NOTIFY * HTTP/1.1
9	0.317060900	172.23.16.70	239.255.255.250	SSDP	450	NOTIFY * HTTP/1.1
10	0.317065100	172.23.16.70	239.255.255.250	SSDP	489	NOTIFY * HTTP/1.1
11	0.317066000	172.23.16.70	239.255.255.250	SSDP	450	NOTIFY * HTTP/1.1
12	0.317068400	172.23.16.70	239.255.255.250	SSDP	503	NOTIFY * HTTP/1.1
13	0.317069200	172.23.16.70	239.255.255.250	SSDP	521	NOTIFY * HTTP/1.1
14	0.317070700	172.23.16.70	239.255.255.250	SSDP	505	NOTIFY * HTTP/1.1
15	0.317071600	172.23.17.236	255.255.255.255	UDP	56	59783 → 3289 Len=14
16	0.317072200	172.23.17.236	172.23.31.255	UDP	56	59784 → 3289 Len=14

> Frame 1: Packet, 77 bytes on wire (616 bits), 77 bytes captured (616 bits) on interface \Device\NPF...
 > Ethernet II, Src: AzureWaveTec_d4:3d:34 (9c:c7:d3:d4:3d:34), Dst: IPv4mcast_fb (01:00:5e:00:00:fb)
 > Internet Protocol Version 4, Src: 172.23.22.137, Dst: 224.0.0.251
 > User Datagram Protocol, Src Port: 5353, Dst Port: 5353

Source Port: 5353
 Destination Port: 5353
 Length: 43
 Checksum: 0x4a3f [unverified]
 [Checksum Status: Unverified]
 [Stream index: 0]
 [Stream Packet Number: 1]
 > [Timestamps]
 UDP payload (35 bytes)
 > Multicast Domain Name System (query)

tcp

No.	Time	Source	Destination	Protocol	Length	Info
43	0.614704800	20.195.84.7	172.23.18.113	TLSv1.2	81	Application Data
44	0.669641500	172.23.18.113	20.195.84.7	TCP	54	52816 → 443 [ACK] Seq=1 Ack=28 Win=253 Len=0
263	6.218396200	172.23.18.113	4.150.223.106	TCP	54	57149 → 443 [FIN, ACK] Seq=1 Ack=1 Win=1021 Len=0
289	6.554504900	4.150.223.106	172.23.18.113	TCP	54	443 → 57149 [FIN, ACK] Seq=1 Ack=2 Win=16385 Len=0
290	6.554587200	172.23.18.113	4.150.223.106	TCP	54	57149 → 443 [ACK] Seq=2 Ack=2 Win=1021 Len=0
350	7.989541100	140.82.113.26	172.23.18.113	TLSv1.2	84	Application Data
351	7.989897400	172.23.18.113	140.82.113.26	TLSv1.2	84	Application Data
393	8.318644300	140.82.113.26	172.23.18.113	TCP	58	443 → 51550 [ACK] Seq=27 Ack=31 Win=74 Len=0
394	8.756219100	172.23.18.113	20.195.84.7	TLSv1.2	85	Application Data
395	8.807449300	20.195.84.7	172.23.18.113	TCP	54	443 → 52816 [ACK] Seq=28 Ack=32 Win=627 Len=0
470	11.065071200	52.110.15.146	172.23.18.113	TLSv1.2	87	Application Data
471	11.116918600	172.23.18.113	52.110.15.146	TCP	54	49242 → 443 [ACK] Seq=1 Ack=34 Win=255 Len=0
582	12.891968400	172.23.18.113	23.103.239.161	TLSv1.2	82	Application Data
583	12.919416700	23.103.239.161	172.23.18.113	TCP	54	443 → 49231 [ACK] Seq=1 Ack=29 Win=252 Len=0
584	13.189167900	172.23.18.113	74.125.24.188	TCP	55	60382 → 5228 [ACK] Seq=1 Ack=1 Win=253 Len=1
585	13.231692100	74.125.24.188	172.23.18.113	TCP	70	5228 → 60382 [ACK] Seq=1 Ack=2 Win=1047 Len=0 SLE=1 SRE=2

> Frame 43: Packet, 81 bytes on wire (648 bits), 81 bytes captured (648 bits) on interface \Device\...
 > Ethernet II, Src: Cisco_6d:bc:f7 (24:d5:e4:6d:bc:f7), Dst: Intel_6d:e9:df (2c:7b:a0:6d:e9:df)
 > Internet Protocol Version 4, Src: 20.195.84.7, Dst: 172.23.18.113
 > Transmission Control Protocol, Src Port: 443, Dst Port: 52816, Seq: 1, Ack: 1, Len: 27

Source Port: 443
 Destination Port: 52816
 [Stream index: 0]
 [Stream Packet Number: 1]
 > [Conversation completeness: Incomplete (12)]
 [TCP Segment Len: 27]
 Sequence Number: 1 (relative sequence number)
 Sequence Number (raw): 1728017554
 [Next Sequence Number: 28 (relative sequence number)]
 Acknowledgment Number: 1 (relative ack number)
 Acknowledgment number (raw): 3462875793
 0101 = Header Length: 20 bytes (5)
 > Flags: 0x018 (PSH, ACK)

24. Network layer protocol header analysis using Wire shark – SMTP and

ICMP.

The image shows a Windows command prompt window and a Wireshark network capture. The command prompt shows a successful ping to 8.8.8.8. The Wireshark capture shows an SMTP session between a client and a server.

Command Prompt Output:

```
Microsoft Windows [Version 10.0.26200.9168]
(c) Microsoft Corporation. All rights reserved.

C:\Users\reddy>ping 8.8.8.8

Pinging 8.8.8.8 with 32 bytes of data:
Reply from 8.8.8.8: bytes=32 time=9ms TTL=117
Reply from 8.8.8.8: bytes=32 time=10ms TTL=117
Reply from 8.8.8.8: bytes=32 time=10ms TTL=117
Reply from 8.8.8.8: bytes=32 time=12ms TTL=117

Ping statistics for 8.8.8.8:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 9ms, Maximum = 12ms, Average = 10ms

C:\Users\reddy>
```

Wireshark Capture:

The Wireshark capture shows an SMTP session. The first packet is a DNS query for mail.patriots.in. The second packet is a DNS response. The third packet is a TCP SYN packet from 10.10.1.4 to 74.53.140.153 on port 25. The fourth packet is a TCP ACK packet from 74.53.140.153 to 10.10.1.4. The fifth packet is a SMTP EHLO packet from 10.10.1.4 to 74.53.140.153. The sixth packet is a SMTP 220 response from 74.53.140.153 to 10.10.1.4. The seventh packet is a SMTP AUTH LOGIN packet from 10.10.1.4 to 74.53.140.153. The eighth packet is a SMTP 334 response from 74.53.140.153 to 10.10.1.4. The ninth packet is a SMTP User packet from 10.10.1.4 to 74.53.140.153. The tenth packet is a SMTP 235 response from 74.53.140.153 to 10.10.1.4. The eleventh packet is a SMTP MAIL FROM packet from 10.10.1.4 to 74.53.140.153.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.10.1.4	10.10.1.1	DNS	76	Standard query 0x7956 A mail.patriots.in
2	0.034025	10.10.1.1	10.10.1.4	DNS	142	Standard query response 0x7956 A mail.patriots.in CNAME patriots.in A 74.53.140.153 NS ns2.patriots.in NS ns1.patriots.in
3	0.036986	10.10.1.4	74.53.140.153	TCP	62	1470 → 25 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK_PERM
4	0.383936	74.53.140.153	10.10.1.4	TCP	62	25 → 1470 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460 SACK_PERM
5	0.383968	10.10.1.4	74.53.140.153	TCP	54	1470 → 25 [ACK] Seq=1 Ack=1 Win=65535 Len=0
6	0.727603	74.53.140.153	10.10.1.4	SMTP	235	S: 220-xc90.websitewelcome.com ESMTP Exim 4.69 #1 Mon, 05 Oct 2009 01:05:54 -0500 We do not authorize the use of this
7	0.732749	10.10.1.4	74.53.140.153	SMTP	63	C: EHLO GP
8	1.073326	74.53.140.153	10.10.1.4	TCP	60	25 → 1470 [ACK] Seq=182 Ack=10 Win=5840 Len=0
9	1.074123	74.53.140.153	10.10.1.4	SMTP	191	S: 250-xc90.websitewelcome.com Hello GP [122.162.143.157] SIZE 52428800 PIPELINING AUTH PLAIN LOGIN STARTTLS
10	1.076669	10.10.1.4	74.53.140.153	SMTP	66	C: AUTH LOGIN
11	1.419021	74.53.140.153	10.10.1.4	SMTP	72	S: 334 VXNlc5ShbWU6
12	1.419595	10.10.1.4	74.53.140.153	SMTP	84	C: User: Z3VycG9ydG9wQHBhdHJpb3R2Imlu
13	1.761484	74.53.140.153	10.10.1.4	SMTP	72	S: 334 UGFzc3dvcmQ6
14	1.762058	10.10.1.4	74.53.140.153	SMTP	72	C: Pass: cHhuanF0DEyHw==
15	2.121738	74.53.140.153	10.10.1.4	SMTP	84	S: 235 Authentication succeeded
16	2.122354	10.10.1.4	74.53.140.153	SMTP	90	C: MAIL FROM: <gurpartap@patriots.in>

Frame 1: Packet, 76 bytes on wire (608 bits), 76 bytes captured (608 bits) on interface 0

Ethernet II, Src: CradlePoint_3c:17:c2 (00:e0:1c:3c:17:c2), Dst: Netgear_d9:81:60 (00:1f:33:d9:81:60)

Internet Protocol Version 4, Src: 10.10.1.4, Dst: 10.10.1.1

User Datagram Protocol, Src Port: 56166, Dst Port: 53

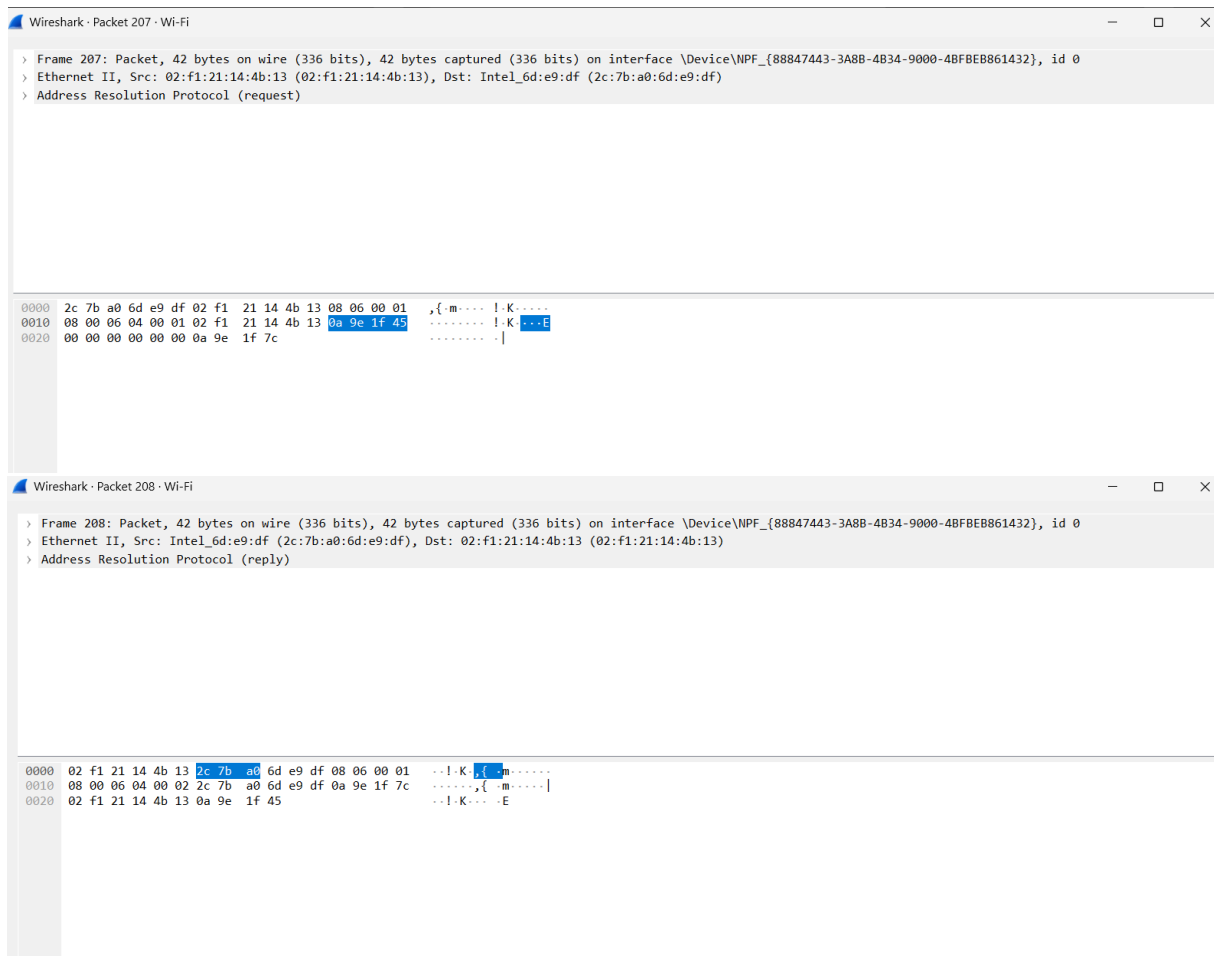
Domain Name System (query)

Packet Details:

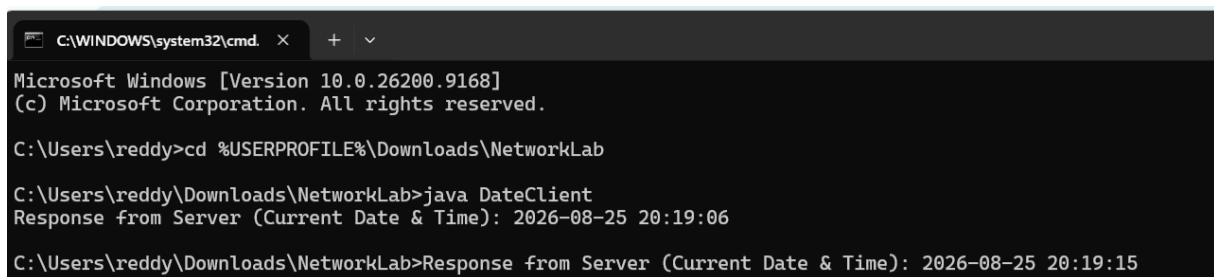
- Standard query type: A
- Question: mail.patriots.in
- Answer: patriots.in A 74.53.140.153
- NS: ns2.patriots.in, ns1.patriots.in

CO4

25. Network layer protocol header analysis using Wire shark – ARP and HTTP.



26. Implementation of date and time display from client to server using TCP sockets in java/C.



CO2

27. Implementation of a DNS server and client in java/C using UDP sockets. CO3

```
C:\WINDOWS\system32\cmd. X + v
Microsoft Windows [Version 10.0.26200.9168]
(c) Microsoft Corporation. All rights reserved.

C:\Users\reddy>cd %USERPROFILE%\Downloads\NetworkLab

C:\Users\reddy\Downloads\NetworkLab>javac DNSServer.java DNSClient.java

C:\Users\reddy\Downloads\NetworkLab>java DNSServer
Server tracking error: Address already in use: bind

C:\Users\reddy\Downloads\NetworkLab>cd %USERPROFILE%\Downloads\NetworkLab

C:\Users\reddy\Downloads\NetworkLab>javac DNSServer.java DNSClient.java

C:\Users\reddy\Downloads\NetworkLab>java DNSServer
DNS Server is running on port 6363... Waiting for queries.
Received Query for Domain: google.com
|
```

```
C:\WINDOWS\system32\cmd. X + v
Microsoft Windows [Version 10.0.26200.9168]
(c) Microsoft Corporation. All rights reserved.

C:\Users\reddy>cd %USERPROFILE%\Downloads\NetworkLab

C:\Users\reddy\Downloads\NetworkLab>java DNSClient
Enter domain name to resolve (e.g., google.com): google.com
DNS Server Response (Resolved IP): 142.250.190.46

C:\Users\reddy\Downloads\NetworkLab>
```

28. Developing a client that contacts a given DNS server to resolve a given hostname in java/C.

CO4

```
C:\WINDOWS\system32\cmd. x + v
Microsoft Windows [Version 10.0.26200.9168]
(c) Microsoft Corporation. All rights reserved.

C:\Users\reddy>cd %USERPROFILE%\Downloads\NetworkLab

C:\Users\reddy\Downloads\NetworkLab>javac DNSQueryClient.java

C:\Users\reddy\Downloads\NetworkLab>java DNSQueryClient
Enter domain hostname to resolve (e.g., example.com): google.com

Transmitting UDP frame to Authoritative Server at 8.8.8.8:53...
Payload frame successfully received back. Decoding binary message bytes...
Validated Server Transaction ID: 0x1234

=== SUCCESSFUL HOST RESOLUTION ===
Resolved IPv4 Endpoint Address: 142.251.223.238

C:\Users\reddy\Downloads\NetworkLab>
```

29. Creating the applications using TCP echo server and client in java/C. CO4

```
C:\WINDOWS\system32\cmd. x + v
Microsoft Windows [Version 10.0.26200.9168]
(c) Microsoft Corporation. All rights reserved.

C:\Users\reddy>cd %USERPROFILE%\Downloads\NetworkLab

C:\Users\reddy\Downloads\NetworkLab>javac EchoServer.java EchoClient.java

C:\Users\reddy\Downloads\NetworkLab>java EchoServer
Echo Server launched. Listening on port 7070...

Client established connection from IP: /127.0.0.1
Received from Client: [Attempting connection to Echo Server at 127.0.0.1:7070...]
Received from Client: [Connection established! Type messages below (type 'exit' to quit).]
Received from Client: [Client Space > Hello Network World!]
Received from Client: [Server Echo > Hello Network World!]
Received from Client: [Client Space > Socket Programming is easy]
Received from Client: [Server Echo > Socket Programming is easy]
Received from Client: [Client Space > exit]
Received from Client: [Server Echo > exit]
Received from Client: [Terminating client socket session connection gracefully.]
Received from Client: [Echo Server launched. Listening on port 7070...]
Received from Client: []
Received from Client: [Client established connection from IP: /127.0.0.1]
Received from Client: [Received from Client: [Hello Network World!]]
Received from Client: [Received from Client: [Socket Programming is easy]]
Received from Client: [Received from Client: [exit]]
Received from Client: [Client closed connection session.]
|
```

```

C:\WINDOWS\system32\cmd. x + v
Client Space > Socket Programming is easy
Server Echo > Socket Programming is easy
Client Space > exit
Server Echo > exit
Terminating client socket session connection gracefully.
Server Echo > Attempting connection to Echo Server at 127.0.0.1:7070...
Client Space > Server Echo > Connection established! Type messages below (type 'exit' to quit)
Client Space > Server Echo > Client Space > Hello Network World!
Client Space > Server Echo > Server Echo > Hello Network World!
Client Space > Server Echo > Client Space > Socket Programming is easy
Client Space > Server Echo > Server Echo > Socket Programming is easy
Client Space > Server Echo > Client Space > exit
Client Space > Server Echo > Server Echo > exit
Client Space > Server Echo > Terminating client socket session connection gracefully.
Client Space > Echo Server launched. Listening on port 7070...

Client established connection from IP: /127.0.0.1
Received from Client: [Hello Network World!]
Received from Client: [Socket Programming is easy]
Received from Client: [exit]
Client closed connection session.
Server Echo > Echo Server launched. Listening on port 7070...
Client Space > Server Echo >
Client Space > Server Echo > Client established connection from IP: /127.0.0.1
Client Space > Server Echo > Received from Client: [Hello Network World!]
Client Space > Server Echo > Received from Client: [Socket Programming is easy]
Client Space > Server Echo > Received from Client: [exit]
Client Space > Server Echo > Client closed connection session.
Client Space > |

```

30. Creating the applications using TCP chat client and chat server in java/C. CO3

```

Error: Could not find or load main class ChatServer
Caused by: java.lang.ClassNotFoundException: ChatServer

C:\Users\reddy\Downloads\NetworkLab>cd %USERPROFILE%\Downloads\NetworkLab

C:\Users\reddy\Downloads\NetworkLab>javac ChatServer.java ChatClient.java

C:\Users\reddy\Downloads\NetworkLab>java ChatServer
Chat Server is launching on port 8080...
Log: [Alice] joined from IP: /127.0.0.1
[Alice]: java ChatClient
[Alice]: cd %USERPROFILE%\Downloads\NetworkLab
[Alice]: hi
[Alice]: how are you?
|

```

31. Implementing ARP protocols in java/C. CO5

```

Microsoft Windows [Version 10.0.26200.9168]
(c) Microsoft Corporation. All rights reserved.

C:\Users\reddy>cd Downloads\NetworkLab

C:\Users\reddy\Downloads\NetworkLab>javac ARPCClient.java

C:\Users\reddy\Downloads\NetworkLab>java ARPCClient
Enter IP Address: 10.10.10.1
The corresponding MAC Address is: MAC Address Not Found

C:\Users\reddy\Downloads\NetworkLab>java ARPCClient
Enter IP Address: 192.168.1.10
The corresponding MAC Address is: 00-11-22-33-44-55

C:\Users\reddy\Downloads\NetworkLab>

```

32. Implementation of Bit stuffing mechanism using C. CO5

```

C:\WINDOWS\system32\cmd. x + v

(c) Microsoft Corporation. All rights reserved.

C:\Users\reddy>cd C:\Users\reddy\Downloads\NetworkLab

C:\Users\reddy\Downloads\NetworkLab>javac BitStuffing.java
BitStuffing.java:52: error: cannot find symbol
    Scanner scanner = new Scanner(System.?n == null ? System.in : System.in);
                                      ^
    symbol:   variable ?n
    location: class System
1 error

C:\Users\reddy\Downloads\NetworkLab>java BitStuffing
Error: Could not find or load main class BitStuffing
Caused by: java.lang.ClassNotFoundException: BitStuffing

C:\Users\reddy\Downloads\NetworkLab>javac BitStuffing.java

C:\Users\reddy\Downloads\NetworkLab>java BitStuffing
Enter the binary data (0s and 1s): 1010

--- Transmitter Side ---
Original Data: 1010
Stuffed Data : 01111110 1010 01111110 (with flags)

--- Receiver Side ---
Destuffed Data: 1010

C:\Users\reddy\Downloads\NetworkLab>

```

33. Implementing the applications using TCP file transfer in java/C. CO5

```

C:\WINDOWS\system32\cmd.  X  +  v

Microsoft Windows [Version 10.0.26200.9168]
(c) Microsoft Corporation. All rights reserved.

C:\Users\reddy>cd C:\Users\reddy\Downloads\NetworkLab

C:\Users\reddy\Downloads\NetworkLab>javac FileClient.java

C:\Users\reddy\Downloads\NetworkLab>java FileClient
Enter the exact filename to download: test.txt
ERROR: File Not Found

C:\Users\reddy\Downloads\NetworkLab>javac FileClient.java

C:\Users\reddy\Downloads\NetworkLab>java FileClient
Enter the exact filename to download: test.txt
File found! Size: 25 bytes. Downloading...
Download complete! Saved as: downloaded_test.txt

C:\Users\reddy\Downloads\NetworkLab>

```

34. Implementing the simulation of error correction code - CRC in java/C. CO5

```

C:\WINDOWS\system32\cmd.  X  +  v

Microsoft Windows [Version 10.0.26200.9168]
(c) Microsoft Corporation. All rights reserved.

C:\Users\reddy>cd C:\Users\reddy\Downloads\NetworkLab

C:\Users\reddy\Downloads\NetworkLab>javac CRCSimulation.java

C:\Users\reddy\Downloads\NetworkLab>java CRCSimulation
Enter original Data Bits (e.g., 1001): 100100
Enter Generator Polynomial Bits (e.g., 1011): 1101

--- Transmitter Side ---
Generated Checksum : 001
Final Codeword Sent: 100100001

--- Receiver Side Simulation ---
Enter received Codeword (or type exact codeword to simulate no error): |

```

35. Implementing the sliding window protocol in java/C.

```
C:\WINDOWS\system32\cmd. X + v
Microsoft Windows [Version 10.0.26200.9168]
(c) Microsoft Corporation. All rights reserved.

C:\Users\reddy>cd C:\Users\reddy\Downloads\NetworkLab

C:\Users\reddy\Downloads\NetworkLab>javac SlidingWindow.java

C:\Users\reddy\Downloads\NetworkLab>java SlidingWindow
Enter the total number of frames to send: 5
Enter the Window Size: 3

--- Starting Sliding Window Transmission ---

Sender: Sending Frames:
-> Frame 1
-> Frame 2
-> Frame 3

Enter the last successfully acknowledged frame number (or 0 for timeout/lost ACK): |
```